

Performance Testing

Date	03 October 2026
Team ID	4
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (load testing), pytest (functional), server timing logs (real Gemini latency)
Type of Testing	Load testing (50 concurrent users) + real-API latency measurement
Target Module / API	GET /, GET /api/health, GET /nutrition-tip, POST /generate-plan
Test Environment	Local: Windows laptop, Uvicorn, SQLite (WAL mode); load test in demo mode, latency test against the live Gemini API
Test Date	03 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Mixed load: home page, health check, nutrition tip and full plan generation + DB write (demo mode)	50 (spawn rate 10/s)	60	Avg < 2 s, max < 5 s, error rate < 1%
2	Concurrent POST /generate-plan writes to SQLite	50	60 (within scenario 1)	No 'database is locked' errors
3	Real Gemini: plan generation, feedback revision, tip (single user)	1	Per request	Valid 7-day plan; latency recorded
4	Primary model overloaded (HTTP 503) → automatic fallback model	1	Per request	Request still succeeds; nothing saved on failure

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	5 ms	Pass	Application stack (FastAPI + Jinja2 + SQLite), demo mode
2	Response Time (Max)	< 5 seconds	357 ms	Pass	Under 50 concurrent users
3	Throughput (Req/sec)	—	24.5 req/s	Pass	1,450 requests in 59 s (Locust HTML report)
4	Error Rate	< 1%	0% (0 / 1,450)	Pass	WAL mode + busy timeout prevented lock errors
5	CPU Utilization	< 80%	Not captured	N/A	Not recorded by Locust in this run
6	Memory Utilization	< 80%	Not captured	N/A	Not recorded by Locust in this run

Real Gemini API latency (live run)

Operation	gemini-3.8-flash (primary)	gemini-3.5-flash-lite (fallback)	Primary overloaded → fallback
Plan generation	14.3 s	4.3 – 5.4 s	18 – 20 s
Feedback revision	12.8 s	3.8 – 5.5 s	17 – 18 s
Nutrition tip	—	≈ 1 s (7.2 s on first call)	—

Step 4: Observations & Analysis

- The application itself is fast: under 50 concurrent users it averaged 5 ms with zero failures. The load test ran in demo mode on purpose, so it measures FastAPI, Jinja2 and SQLite rather than Google's servers, and does not use up the free-tier API quota.
- End-to-end generation time is dominated by the Gemini model: 4–6 s on the light model and 13–15 s on gemini-3.8-flash. The UI shows a loading state during generation.
- During testing Google returned HTTP 503 "high demand" for gemini-3.8-flash on most calls. With one retry only, 4 of 6 requests failed (safely: friendly message, nothing saved). After adding an automatic fallback model, 5 of 5 requests succeeded.
- SQLite WAL mode and a busy timeout were enabled after early concurrent-write tests, giving 0% errors under load.
- Recommendation: for live demos, use the light model as primary for faster, more reliable responses.

Step 5: Screenshots / Evidence

The full Locust HTML report (reports/perf.html), CSV statistics (reports/perf_*.csv), summary (reports/perf_summary.md) and server timing logs are in the reports/ folder at the repository root.

Locust Test Report

During: 10/3/2026, 11:45:24 AM - 10/3/2026, 11:46:23 AM (59 seconds)
Target Host: http://127.0.0.1:8000
Script: locustfile.py

Request Statistics



Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /	557	0	3.74	2	58	4947	9.42	0
GET	GET /api/health	380	0	4.02	2	59	138	6.43	0
GET	GET /nutrition-tip	341	0	3.13	1	23	260.35	5.77	0
POST	POST /generate-plan	172	0	16.47	7	357	4605.11	2.91	0
Aggregated		1450	0	5.18	1	357	2543.98	24.53	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /	3	3	3	4	5	7	19	58
GET	GET /api/health	3	3	3	4	5	12	57	59
GET	GET /nutrition-tip	2	3	3	3	4	6	16	23
POST	POST /generate-plan	11	11	11	12	21	39	280	360
Aggregated		3	3	4	5	11	13	39	360

Figure 1: Locust report — request and response-time statistics (50 users, 59 s, 0 failures)

Charts

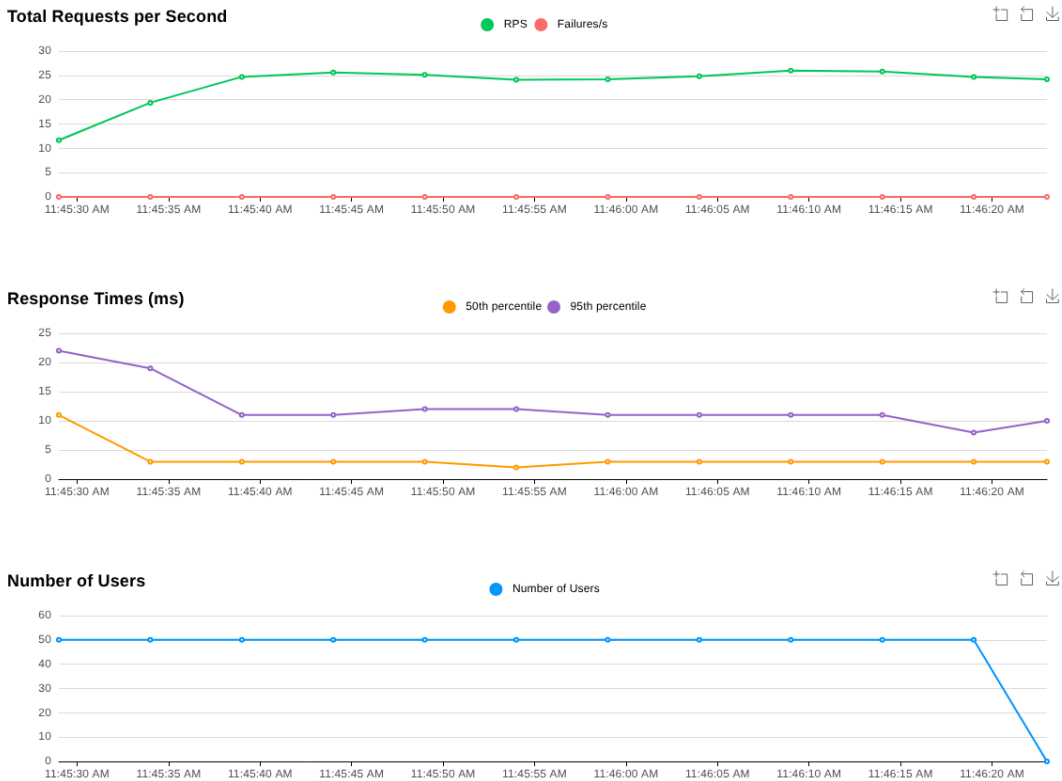


Figure 2: Locust report — requests per second, response-time percentiles and user count

The CSV summary (reports/perf_summary.md) counted 1,433 requests because Locust's CSV writer stopped slightly before the HTML report at shutdown on Windows; both show 0 failures and the same response times.